

Towards a Linear-Algebraic Hypervisor

Breandan Considine

Abstract

Many techniques in program synthesis, superoptimization, and array programming require parallel rollouts of general-purpose programs. GPUs, while capable targets for domain-specific parallelism, are traditionally underutilized by such workloads. Motivated by this opportunity, we introduce a pleasingly parallel virtual machine and benchmark its performance by evaluating millions of concurrent array programs, observing speedups up to 147× relative to serial evaluation.

1 Introduction

An *abstract machine* is a sequential model of computation for analyzing the complexity of algorithms, which, unlike the Turing machine, more closely characterizes the operational behavior of modern computers. Elgot and Robinson [4] give an early example of such a machine, known as the random-access stored program (RASP) machine, equipped with a small set of atomic instructions and a shared address space storing both program and data. Cook and Reckhow [3] later present a simplified RASP and introduce an equivalent model which isolates its program from its data in memory, known as the random-access machine (RAM). An implementation of an abstract machine is known as a *virtual machine* (VM) and a *hypervisor* is a virtualization environment that simulates multiple VMs on a single physical machine, such as a GPU.

1.1 RASP model

A RASP machine is a 4-tuple $\mathcal{R} = \langle C, c_0, \rightarrow, F \rangle$ consisting of a configuration space, $C = \mathbb{N} \times \mathbb{Z} \times \mathbb{Z}^{\mathbb{N}} \times \mathbb{Z}^* \times \mathbb{Z}^*$, which we will write as $\langle i, a, M, u, y \rangle : C$, with i being the *instruction counter*, a being the *accumulator*, M being the *memory*, u being the *unread input*, and y being the *output*. A RASP program is a vector $P : (\mathbb{Z} \times \mathbb{Z})^m$ that is packed with an *input* $x : \mathbb{Z}^*$, into an *initial configuration*, $c_0(P, x)$ as $\langle 0, 0, M'_{0..2m} \leftarrow P, x0, \varepsilon \rangle$. Letting $\langle o, j \rangle = \langle M_i, M_{i+1} \rangle$ denote the *opcode* and *operand* respectively, the one-step transition function $\rightarrow : C^C$ is

$$\langle i, a, M, u, y \rangle \rightarrow \begin{cases} \langle i+2, j, M, u, y \rangle, & o = 1_{\text{LOD}}, \\ \langle i+2, a + M_j, M, u, y \rangle, & o = 2_{\text{ADD}}, \\ \langle i+2, a - M_j, M, u, y \rangle, & o = 3_{\text{SUB}}, \\ \langle i+2, a, M_j \leftarrow a, u, y \rangle, & o = 4_{\text{STO}}, \\ \langle j, a, M, u, y \rangle, & o = 5_{\text{BPA}} \wedge a > 0, \\ \langle i+2, a, M, u, y \rangle, & o = 5_{\text{BPA}} \wedge a \leq 0, \\ \langle i+2, a, M_j \leftarrow \sigma, v, y \rangle, & o = 6_{\text{RD}} \wedge u = \sigma v, \\ \langle i+2, a, M, u, y \cdot M_j \rangle, & o = 7_{\text{PRI}}, \\ \langle i, a, M, u, y \rangle, & \text{otherwise.} \end{cases}$$

The RASP machine's halting configurations, $F \subseteq C$, are characterized by the fixed points, $F = \{c \in C \mid c \rightarrow c\}$.

2 Method

We will consider a parallel RASP variant loosely equivalent to the PRAM [5] model, except each machine stores its program in memory and all machines occupy constant space.

Concretely, we lift the transition function to a vectorized transition map, $\Phi : C^d \rightarrow C^d$ whose iterates are given by $\Phi^{i+1} = \Phi \circ \Phi^i$, so the resulting configuration after i steps is $c_i = \Phi^i(c_0)$. Thus, for a vector of programs $\mathbf{P}$ and inputs $\mathbf{x}$, we will write $c_i(\mathbf{P}, \mathbf{x}) := \Phi^i(c_0(\mathbf{P}, \mathbf{x}))$ for the evolution of n independent machines, each with static memory. Since memory is statically allocated, each stepwise transition is a Boolean function on finitely many words, hence Φ admits a unique multilinear polynomial representation over $\mathbb{F}_2$ [8].

2.1 Word RASP

Fix $w, m, n, \ell, s \in \mathbb{N}$, and assume a fixed-length word, $\mathbb{W} = \mathbb{F}_2^w$. Then, for a program $P : \mathbb{W}^{2m \leq n}$, define the word RASP as $\mathcal{R}^{(w, n, \ell, s)} = \langle C_w, c_0, \rightarrow, F \rangle$ with a finite configuration space,

$$C_w = \mathbb{W} \times \mathbb{W} \times \mathbb{W}^n \times \mathbb{W}^{\ell+1} \times \mathbb{W}^{s+1} = \mathbb{F}_2^{w(n+\ell+s+4)}, \\ c = \langle i, a, M, u, y \rangle$$

where the initial configuration on input x is given by

$$c_0(P, x) = \langle 0_w, 0_w, P 0_w^{n-2m}, x 0_w^{\ell-|x|}, 0_w^{s+1} \rangle.$$

We write $\oplus, \otimes : \mathbb{W} \times \mathbb{W} \rightarrow \mathbb{W}$ for addition and multiplication (mod 2^w) and assume a circular memory addressing scheme,

$$\langle o, j \rangle := \langle M_{i \bmod n}, M_{(i \oplus 1) \bmod n} \rangle.$$

Now, let δ_{jk} be the Kronecker delta and define the operators,

$$M_j \leftarrow a := [\delta_{jk} a + (1 - \delta_{jk}) M_k]_{k=0}^{n-1}, \\ u^\bullet := u_{\min(u_0+1, \ell)}, \quad u^\circ := u_0 \leftarrow \min(u_0 + 1, \ell), \\ y :: z := \begin{cases} (y_{y_0+1} \leftarrow z)_0 \leftarrow y_0 + 1, & y_0 < s, \\ y, & \text{otherwise.} \end{cases}$$

The one-step transition operator takes $\langle i, a, M, u, y \rangle$ to

$$\begin{cases} \langle i \oplus 2_w, j, M, u, y \rangle, & o = 1_{\text{LOD}}, \\ \langle i \oplus 2_w, a \oplus M_{j \bmod n}, M, u, y \rangle, & o = 2_{\text{ADD}}, \\ \langle i \oplus 2_w, a \otimes M_{j \bmod n}, M, u, y \rangle, & o = 3_{\text{MUL}}, \\ \langle i \oplus 2_w, a, M_{j \bmod n} \leftarrow a, u, y \rangle, & o = 4_{\text{STO}}, \\ \langle j, a, M, u, y \rangle, & o = 5_{\text{BPNZ}} \wedge a \neq 0, \\ \langle i \oplus 2_w, a, M, u, y \rangle, & o = 5_{\text{BPNZ}} \wedge a = 0, \\ \langle i \oplus 2_w, a, M_{j \bmod n} \leftarrow u^\bullet, u^\circ, y \rangle, & o = 6_{\text{RD}} \wedge u_0 < \ell, \\ \langle i \oplus 2_w, a, M, u, y :: M_{j \bmod n} \rangle, & o = 7_{\text{PRI}}, \\ \langle i, a, M, u, y \rangle, & \text{otherwise.} \end{cases}$$

Once again, the word RASP machine's halting configurations are characterized by the fixed points, $F = \{c \in C_w \mid c \rightarrow c\}$.

Let $\Phi_w : C_w \rightarrow C_w$ denote the resulting coordinatewise polynomial, so $c \rightarrow c' \iff \Phi(c) = c'$. Alternatively, this can be written as a total deterministic transition function,

$$\Phi_w(c) = \bigoplus_{\psi} \eta_{\psi}(c) \Phi_{\psi}(c), \quad \bigoplus_{\psi} \eta_{\psi}(c) = 1,$$

where ψ ranges over the guarded instruction cases, η_{ψ} is a multilinear indicator, and Φ_{ψ} is the associated branch update.

2.2 Heapless array lowering

Consider a small array language with the following terms:

```

F ::= fun f0 ( ipt : W ^ N ) -> W ^ N { B }
N ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
B ::= S | S ; B | hlt
S ::= P = E | ife G { B } { B } | whl G { B }
Z ::= 0 | N
G ::= ipt [ Z ] | scr [ Z ] | opt [ Z ]
P ::= scr [ Z ] | opt [ Z ]
E ::= G + G | G * G | G + Z | G * Z | G | Z
    
```

Reserve disjoint memory regions $p_{0\dots l-1}$, $q_{0\dots e-1}$, and $r_{0\dots y-1}$ for arrays $\text{ipt} : \mathbb{W}^{l \leq \ell}$, $\text{opt} : \mathbb{W}^{e \leq s}$ and $\text{scr} : \mathbb{W}^{y \leq \mu}$, with $(5 + \ell + s + \mu + 2m) \leq n$ and interpret array indexing modulo declared length, so $\llbracket \alpha \rrbracket_z := M_{2m+\llbracket \alpha \rrbracket + (z \bmod |\alpha|)}$. We will now assign a small-step operational semantics to this language:

$$\begin{array}{c}
 \frac{}{\Gamma \vdash B \Rightarrow C} \text{FUN} \quad \frac{}{\vdash \text{fun } f0 \text{ (ipt : } I \wedge N \text{) } \rightarrow I \wedge l \{ B \} \Rightarrow C} \text{PRI } q_l \text{ }^{l-1} \text{ } \text{HLT} \\
 \frac{}{\Gamma \vdash \alpha[z] \Rightarrow_a \text{LOD } 0 \text{ } \text{ADD } \llbracket \alpha \rrbracket_z} \text{GET} \quad \frac{\Gamma \vdash E \Rightarrow_a A}{\Gamma \vdash \alpha[z] = E \Rightarrow A \text{STO } \llbracket \alpha \rrbracket_z} \text{PUT} \\
 \frac{\Gamma \vdash I_1 \Rightarrow_a J_1 \quad \Gamma \vdash I_2 \Rightarrow_a J_2}{\Gamma \vdash I_1 + I_2 \Rightarrow_a J_1 \text{STO } v \text{ } J_2 \text{ADD } v} \oplus \quad \frac{\Gamma \vdash I_1 \Rightarrow_a J_1 \quad \Gamma \vdash I_2 \Rightarrow_a J_2}{\Gamma \vdash I_1 \times I_2 \Rightarrow_a J_1 \text{STO } v \text{ } J_2 \text{MUL } v} \otimes \\
 \frac{\Gamma \vdash I \Rightarrow_a J \quad \Gamma \vdash B_1 \Rightarrow C_1 \quad \Gamma \vdash B_2 \Rightarrow C_2}{\Gamma \vdash \text{ife } I \{ B_1 \} \{ B_2 \} \Rightarrow J \text{BNZ } \ell_b \text{ } C_2 \text{JMP } \ell_e \text{ } \ell_t : C_1 \ell_e :} \text{IFE} \\
 \frac{\Gamma \vdash I \Rightarrow_a J \quad \Gamma \vdash B \Rightarrow C}{\Gamma \vdash \text{whl } I \{ B \} \Rightarrow \ell_h : J \text{BNZ } \ell_b \text{ } \text{JMP } \ell_e \text{ } \ell_b : C \text{JMP } \ell_h \text{ } \ell_e :} \text{WHILE} \\
 \frac{\Gamma \vdash B \Rightarrow C_B \quad \Gamma \vdash S \Rightarrow C_S}{\Gamma \vdash B ; S \Rightarrow C_B C_S} \text{SEQ} \quad \frac{}{\Gamma \vdash \text{hlt} \Rightarrow \text{PRI } q_l \text{ }^{l-1} \text{ } \text{HLT}} \text{HALT}
 \end{array}$$

We write $\Gamma \vdash E \Rightarrow A$ when the instruction sequence A implements E , and $\Gamma \vdash E \Rightarrow_a A$ when A implements E and leaves value a in the accumulator. Let v be a single reserved memory address and finally, desugar $\text{JMP } \ell$ as $\text{LOD } 1 \text{BNZ } \ell$, with HLT being any reserved invalid instruction pair whose execution would trigger a fixed-point halt (e.g., 00).

2.3 Implementation

VMs are implemented using a mainly branchless strategy to reduce warp divergence on the GPU, and host threads are dispatched to VMs using a simple round-robin scheduler. A short supplemental, including hypervisor pseudocode and platform-specific details, may be found in Appendix A.

3 Evaluation

We implement VMs for the word RASP on (1) Kotlin/JVM and (3) Nvidia CUDA with parameters $w = 32$, $n = 250$, $\ell = 10$, $s = 2$, $\tau_{\max} = 10^6$. Then we evaluate each implementation on $d = 8 \times 10^6$ random array programs sampled uniformly from the space of valid code snippets of exactly length $L = 100$ using Considine’s word sampler [2]. In Fig. 1a, we measure runtime across the JVM (running on an Apple M4 Max) and CUDA platforms (on the Nvidia A10G and B200 GPUs), reporting wall-clock time to evaluate up to $\tau_{\max}$ VM steps.

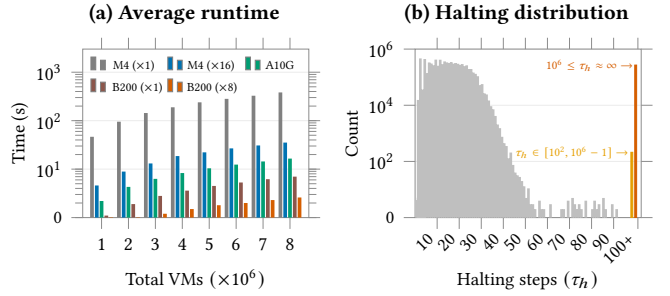


Figure 1. Wallclock and simulation runtime of 8m programs.

In Fig. 1b, we sample $d = 8 \times 10^6$ programs of length $L = 100$ and estimate the halting probability before $\tau_{\max} = 10^6$ steps,

$$\Pr_{P' \sim \mathcal{P}_L} \left(\Phi_w^{\tau_{\max}}(c_0(P' \Rightarrow P, x \sim \mathbb{F}_2^{w \cdot \text{arity}(P')}) \in F \mid L = |P'| \right).$$

We plot the distribution of halting programs by their halting number τ_h , i.e., the least τ such that $\Phi_w^{\tau}(c_0) = \Phi_w^{\tau+1}(c_0)$.

4 Related work

Prior work has explored compiling to transformers [10], though not intended as a practical compiler per se. The most relevant systems literature is Vectorvisor [6], which targets WebAssembly IR, requiring a much larger set of opcodes. Exhaustive search has played a key role in recent empirical mathematics projects [1, 7] which use distributed computing to filter for constructions with favorable properties, but do not explicitly leverage SIMD architectures. We specifically target a much smaller IR and heap space, enabling us to run millions of concurrent VMs. The RASP model’s simplicity, low memory footprint, and proximity to realistic assembly makes it suitable for algorithmic analysis, program synthesis, empirical research, and a variety of pedagogical applications.

5 Conclusion

We have presented a multilinear RASP VM, illustrated its viability by lowering a heapless array language, and used it to estimate the probability of sampling halting programs, demonstrating significant speedups over naïve evaluation. In future work, we intend to use it to accelerate the search for straight-line programs for fast matrix multiplication and context-free grammar parsing. Source code and data for all experiments will be released here: <https://github.com/breandan/lavm>

References

- [1] Justin Blanchard, Daniel Briggs, Konrad Deka, Nathan Fenner, Yannick Forster, Georgi Georgiev, Matthew L House, Rachel Hunter, Maja Kądziołka, Pavel Kropitz, et al. 2025. Determination of the fifth Busy Beaver value. *arXiv preprint arXiv:2509.12337* (2025).
- [2] Brendan Considine. 2025. A word sampler for well-typed functions. arXiv:2512.01036 [cs.PL] <https://arxiv.org/pdf/2512.01036.pdf>
- [3] Stephen A Cook and Robert A Reckhow. 1972. Time-bounded random access machines. In *Proceedings of the fourth annual ACM symposium on Theory of computing*. 73–80.
- [4] Calvin C Elgot and Abraham Robinson. 1964. Random-access stored-program machines, an approach to programming languages. *Journal of the ACM (JACM)* 11, 4 (1964), 365–399.
- [5] Steven Fortune and James Wyllie. 1978. Parallelism in random access machines. In *Proceedings of the tenth annual ACM symposium on Theory of computing*. 114–118.
- [6] Samuel Ginzburg, Mohammad Shahrad, and Michael J Freedman. 2023. VectorVisor: a binary translation scheme for throughput-oriented GPU acceleration. In *2023 USENIX Annual Technical Conference*. 1017–1037.
- [7] Manuel Kauers and Jakob Moosbauer. 2023. Flip graphs for matrix multiplication. In *Proceedings of the 2023 International Symposium on Symbolic and Algebraic Computation*. 381–388.
- [8] Elchanan Mossel, Ryan O'Donnell, and Rocco P Servedio. 2003. Learning juntas. In *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*. 206–212.
- [9] NVIDIA Corporation. 2026. *CUDA Programming Guide*. <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html> Release 13.2.
- [10] Gail Weiss, Yoav Goldberg, and Eran Yahav. 2021. Thinking like transformers. In *International Conference on Machine Learning*. PMLR, 11080–11090.

A Implementation details

We launch W persistent workers, where W is the maximum number of resident threads, and stripe the global configuration vector $\mathbf{c}$, so that worker g visits each VM in its stripe in round-robin order. Each visit advances one live VM for a bounded epoch q , yielding a static schedule that realizes the lifted dynamics up to a finite time horizon, $\Phi^{\tau_{\max}} : C_w^d \rightarrow C_w^d$.

Algorithm 1 Striped round-robin RASP hypervisor

```

1: inputs: Global config  $\mathbf{c}_0 \in C^d$ , epoch  $q$ , step budget  $\tau_{\max}$ 
2:  $\mathbf{c} \leftarrow \mathbf{c}_0$ ,  $W \leftarrow \text{MAXRESIDENTTHREADS}()$ 
3: for  $g = 0$  to  $W - 1$  in parallel do           ▶ Persistent workers
4:   for  $r = 0$  to  $\lceil \tau_{\max}/q \rceil - 1$  do           ▶ Total rounds
5:     for  $k = 0$  to  $\lceil d/W \rceil - 1$  do           ▶ Walk worker stripe
6:        $j \leftarrow g + kW$                        ▶ Current VM index
7:       if  $j < d$  and  $\mathbf{c}[j] \notin F$  then           ▶ Skip halted VMs
8:          $R \leftarrow \text{LOADREGS}(\mathbf{c}[j])$ 
9:         for  $t = 1$  to  $q$  do                       ▶ Bounded epoch
10:          if  $R \in F$  then break
11:           $R \leftarrow \Phi_w(R)$                    ▶ One-step transition
12:           $\mathbf{c}[j] \leftarrow \text{STOREREGS}(R)$ 
13: return  $\mathbf{c}$ 

```

Since CUDA is limited to at most 255 registers per thread [9], we constrain VM memory to $n \lesssim 250$ 32-bit words, however this constraint can be relaxed, VRAM permitting, to 1 kB or higher, albeit at the cost of increased memory traffic. Assuming 256 resident threads per SM, an Nvidia B200 can sustain about 37,888 resident threads at peak theoretical occupancy.

B Heapless busy beavers ($L = 120$)

Listed below are a few of the busiest beavers discovered after conducting a ~ 20 minute search with the JVM running on an M4 ($\times 16$) architecture using the grammar from Sec. 2.2, with the `ipt`, `opt` and `scr` parameters all initialized to 0s.

BB #1 ($\tau_h = 1549$)

```

fun f0(ipt: W ^ 4) -> W ^ 1 {
  opt[0] = 4;
  whl opt[0] {
    opt[0] = opt[0] + opt[0];
    scr[3] = opt[0];
    whl opt[0] {
      whl opt[0] {
        scr[7] = scr[2] * 7;
        opt[0] = 0;
        scr[1] = 9
      }
    }
  };
  opt[0] = scr[2] + 0;
  opt[0] = scr[3] + scr[5]
}

```

BB #2 ($\tau_h = 1272$)

```

fun f0(ipt: W ^ 2) -> W ^ 1 {
  opt[0] = opt[0] + 6;
  whl opt[0] {
    scr[3] = opt[0] + 0;
    if ipt[0] { hlt } {
      scr[8] = 6;
      opt[0] = ipt[0] + 3; scr[0] = ipt[1] * 2;
      opt[0] = scr[3] * 6; scr[6] = scr[0] * 3;
      opt[0] = opt[0]
    }
  }
}

```

BB #3 ($\tau_h = 1255$)

```

fun f0(ipt: W ^ 3) -> W ^ 1 {
  opt[0] = 1;
  whl opt[0] {
    opt[0] = opt[0] + ipt[0];
    if ipt[2] { hlt } {
      scr[4] = 6;
      opt[0] = opt[0] + opt[0]
    }
  };
  scr[6] = scr[8] * 2;
  whl ipt[1] { scr[4] = 3; opt[0] = 8 };
  scr[0] = opt[0] + 9
}

```